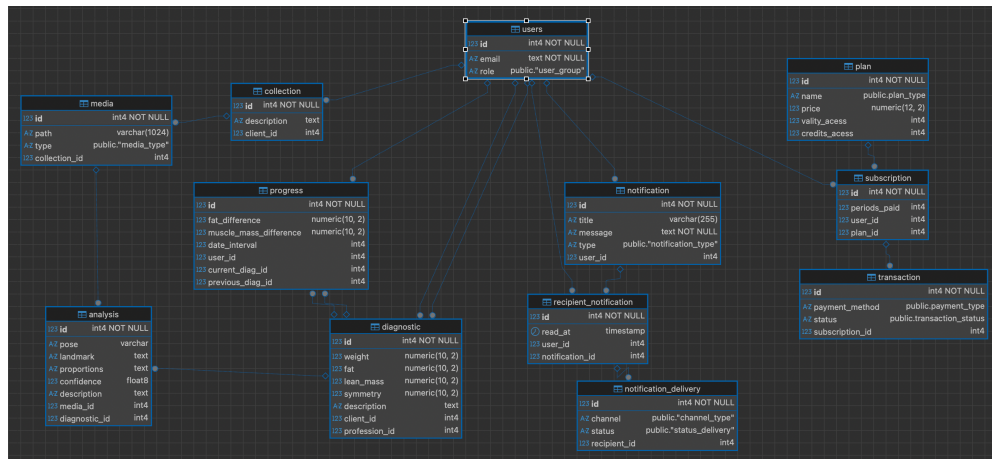


Model - Database v3

Diagrama



alt text

– Nota – v1:

A princípio, a ideia do projeto não precisa de uma estrutura complexa de tabelas (na minha visão), portanto a estrutura idealizada em sala já estava muito boa.

O que busquei pesquisar foi alguma plataforma que tivesse uma proposta de estrutura de banco de dados parecida com a nossa, e cheguei ao design do banco de dados do Instagram, onde você possui posts e cada post possui arquivos de imagem/vídeo, e esses arquivos de mídia possuem comentários.

Por mais que a proposta final seja diferente, a estrutura utilizada é similar ao que buscamos (na minha visão), e portanto acredito que seja um bom exemplo a se seguir.

– Nota – v2:

Foi realizada a implementação inicial do Schema de notificações, estruturado com base nas novas referencias listadas.

– Nota – v3:

Foi adicionado o design inicial de inscrições nos planos. Não identifiquei tanto conteúdo assim para este tópico, mas o artigo adicionado abaixo ajudou a idealizar o restante por conta própria.

Tables Explanations

- Users

Tabela de usuários e seus papéis

```
create type user_group as enum (  
    'client', 'admin', 'professional'  
);  
  
create table users (  
    id INT generated always as identity primary key,  
    email TEXT unique not null,  
    role user_group  
  
    -- demais atributos a serem adicionados  
);
```

- Collection

Grupo de fotos e vídeos enviados pelo usuário (pode ser imaginado como um post no Instagram, onde você pode postar uma simples foto ou fazer um carrossel).

```
create table collection ( -- grupo de arquivos de midia  
    id INT generated always as identity primary key,  
    description TEXT,  
  
    client_id INT references users(id)  
);
```

- Media

Arquivo unitário. Se collection é o grupo de arquivos, aqui ficará apenas a referência de um arquivo.

```
create type media_type as enum (  
    'image', 'video'  
);  
  
create table media ( -- unidade de arquivo de midia  
    id INT generated always as identity primary key,  
    path VARCHAR(1024),  
    type media_type,  
  
    collection_id INT references collection(id)  
);
```

- Analysis

Pense em Analysis como o par da tabela Media. Aqui será feita a análise de cada foto/

vídeo enviado.

```
create table analysis ( -- analise unitaria de cada imagem/video
  id INT generated always as identity primary key,
  pose VARCHAR, -- maybe can be ENUM
  landmark TEXT, -- pontos de referencia gerados pela IA (se necessário)
  proportions TEXT, -- proporcoes do corpo
  confidence float, -- precisao nas analises
  description TEXT,

  media_id INT references media(id),
  diagnostic_id INT references diagnostic(id)
);
```

- Diagnostic

Se a tabela Analysis é o par da tabela Media, a tabela Diagnostic é o par da tabela Collection. Aqui nós iremos pegar todas as análises unitárias feitas para cada arquivo de mídia e gerar um resultado final (diagnóstico).

```
create table diagnostic ( -- diagnostico gerado a partir da
  colecao de analises
  id INT generated always as identity primary key,
  weight numeric(10,2), -- peso
  fat numeric(10,2), -- gordura corporal
  lean_mass numeric(10,2), -- massa magra
  symmetry numeric(10,2),
  description TEXT,

  client_id INT references users(id),
  profession_id INT references users(id)
);
```

- Progress

Tabela que será utilizada para gerar uma timeline do usuário para poder visualizar sua evolução.

```
create table progress ( -- timeline dos diagnosticos \ evolucao do
  usuario
  id INT generated always as identity primary key,
  fat_difference numeric(10,2),
  muscle_mass_difference numeric(10,2),
  date_interval int,

  user_id INT references users(id),
  current_diag_id INT references diagnostic(id),
  previous_diag_id INT references diagnostic(id)
);
```

Notification Schema

- Notification

Estrutura da notificação. O texto que será enviado, título...

```
create type notification_type as enum (  
    'INFO', 'WARNING', 'ERROR'  
);  
  
create table notification (  
    id INT generated always as identity primary key,  
    title VARCHAR(255),  
    message TEXT not null,  
    type notification_type,  
  
    user_id INT references users(id)  
);
```

- Recipient Notification

Referencia sobre destinatario da notificacao

```
create table recipient_notification (  
    id INT generated always as identity primary key,  
    read_at TIMESTAMP null,  
  
    user_id INT references users(id),  
    notification_id INT references notification(id)  
);
```

- Channel

Informacoes sobre a entrega da notificacao. Canal pelo qual foi enviado, status do envio...

```
create type channel_type as enum (  
    'SYSTEM', 'EMAIL'  
);  
  
create type status_delivery as enum (  
    'PENDING', 'FAILED', 'SENT'  
);  
  
create table notification_delivery (  
    id INT generated always as identity primary key,  
    channel channel_type,  
    status status_delivery,  
  
    recipient_id INT references recipient_notification(id)
```